Understanding databases is a deep topic in its own right and beyond the scope of this book. This chapter will give you enough background in relational databases to understand how to use the ORM, but you may want to dig deeper into databases.

4.2 ORM Models

At the heart of an ORM is a mapping between Python classes and tables in a relational database. Unlike Python's dynamic typing, where you can define what kind of data goes in a variable at run time, relational databases are picky about the data types of a column. To define a mapping between a class and a table, you need to be explicit about the data type of the columns. Django's ORM provides a base class you inherit from and a series of fields that specify the types of data for the mapping.

Let's start by defining a musician class that contains the first name, last name, and date of birth of the musician. The first question is: Where do you put this? In chapter 2, when you created a Django app, one of the skeleton files built for you was models.py. This file is where Django expects you to build you ORM models. Currently, you only have the home app that contains general content for your site. Create a new app to contain the data definitions and views for musicians and their bands, and call it *bands*. Remember, you do this with the `manage.py startapp` command:

```
(venv) RiffMates$ python manage.py startapp bands
```

In addition to creating the app, recall that you need to tell Django about it by adding it to the `INSTALLED_APPS` configuration value inside RiffMates/RiffMates/settings.py, like in the following listing.

Listing 4.1 Tell Django about your `bands` app

```
# RiffMates/RiffMates/settings.py
...
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'home',
    'bands',
]
...
```

Add the bands app to the INSTALLED_APPS list.

Running the `startapp` command created skeleton files for you in the bands directory, including the models.py file where you define ORM classes. You tell Django that your class is an ORM model by inheriting from `django.db.models.Model`. The skeleton model.py file imports the `models` package into its namespace for you.

The models package includes both the base class for ORM models and all the data type fields for mapping to database columns. A musician class that includes first name,